

Jogendra Ayatha

Email: jogendraayatha@gmail.com | Phone: +1 972-734-8551 | LinkedIn: linkedin.com/in/aniruddhayatha/

Professional Summary:

- Site Reliability & DevOps Engineer with 10 years of experience building scalable infrastructure, automating cloud deployments, and optimizing **CI/CD** pipelines across **AWS**, **Azure**, and **GCP**. Proven expertise in IaC, container orchestration, observability tooling, and security-compliant infrastructure delivery.
- Hands-on experience in architecting, deploying, and managing **AWS** Cloud platform and its features which includes S3, EC2, VPC, SNS, RDS, CloudWatch, CloudFormation, Autoscaling, IAM and Route53.
- Implemented migration strategies for **Azure** services including Azure storage, IIS, Active Directory, Azure Resource Manager, Azure Storage, Blob Storage, VMs, SQL Database, Azure Functions, Azure Monitor, and Azure Service Bus.
- Extensive experience using **Google Cloud Platform (GCP)** services including Compute Engine, Cloud Functions, Cloud DNS, Cloud Storage and implementing SaaS, PaaS and IaaS concepts through **GCP**.
- Created and maintained highly scalable and fault-tolerant multi-tier **AWS**, **Azure** and **GCP** environments across multiple availability zones using **Terraform** and **CloudFormation**.
- Set up **Kubernetes** (k8s) clusters for running microservices and deployed to production using **Kubernetes** backed Infrastructure. Developed automation for **Kubernetes** clusters using playbooks in **Ansible**. Automated **Kubernetes cluster** management using **Ansible** and **YAML**, deployed **Kubernetes clusters** on **Amazon EC2** instances.
- Successfully migrated infrastructure automation from **Chef** to **Docker** and **Kubernetes**, leveraging containerization and orchestration best practices to enhance scalability, deployment speed, and efficiency across environments.
- Leveraged **Red Hat** Advanced Cluster Management (ACM) to enforce **GitOps** based policy governance and maintain consistent cluster configurations across distributed **AKS** environments.
- Developed and migrated microservices using **AWS** and **Azure** services and applied **Blue/Green** deployment strategies to ensure zero-downtime releases, fast rollback on failure, and enhanced system availability and reliability.
- Worked extensively with **Jenkins** and **Code Pipeline**, **CI/CD** pipeline jobs to create end-to-end automation for building, testing and delivering artifacts, along with troubleshooting build issues during the **Jenkins** build process.
- Developed Microservices, created test cases and worked with various **Python** application development and **CI/CD** pipeline tools including **GitHub**, **Jenkins**, **AWS**, and **Unix** shell scripting.
- Wrote **Groovy** Scripts for Continuous Integration, used **Jenkins** to upload Artifacts to **Nexus** Repository and deployed those artifacts into **Docker** containers. Used **Git**, **Bitbucket** for version control and **Jenkins** for Continuous Integration and Continuous Deployment, ensuring efficient and reliable software delivery.
- Developed and deployed **JAVA** REST APIs via micro-services using **Docker** and **AWS ECS** and followed Application Test Driven Development (ATDD) to create test plans and code coverage tests.
- Automated Build for deploying services in **Docker** environment using **Jenkins**, **SonarQube**, **Gradle**, **Groovy**, **Docker** and **Splunk**. Helped install, configure, and maintain **Unix/Linux** servers on **CentOS**, **Ubuntu**, **Red Hat**.
- Deployed **Dynatrace** OneAgent to monitor containerized applications in **AKS** and **EC2**. Also utilized **Nagios** and **Munin** for legacy system performance monitoring and resource utilization tracking.
- Set up and maintained web servers using **Apache**, **Nginx** and database servers like **MySQL**, **PostgreSQL**, ensuring high availability for business applications across development, staging, and production environments.
- Implemented monitoring solutions using **Prometheus**, **Grafana**, **AppDynamics**, **Datadog**, and **New Relic** to improve observability across **Kubernetes** and cloud environments for enhanced system reliability.

TECHNICAL SKILLS:

Version Control	GIT, GitHub, SVN, Bit Bucket
Scripting	Linux, Python, JavaScript, YAML, Perl, PHP, Bash, Shell, PowerShell, JSON
CI/CD	Jenkins, Code Pipeline, Gitlab, GitHub Actions, Bamboo
Build Tools	MAVEN, Gradle, ANT, Nexus, Artifactory
Container Technologies	Docker, Docker Hub, Kubernetes, Helm
Configuration Management	Chef, Puppet, Ansible, Terraform, CloudFormation
Virtualization Tech	VMware vSphere, VMware Workstation, Oracle Virtual box
Monitoring Tools	Nagios, Prometheus, Splunk, Grafana, OpenTelemetry, Elasticsearch, Logstash, Kibana
APM Tools	Datadog, New Relic, Dynatrace, AppDynamics, SonarQube, Checkmarx
Databases	MY SQL, Oracle 9i/10g/11g, DynamoDB, PostgreSQL
Application Servers	Apache Tomcat, Nginx, Red Hat, WebSphere
Web Servers	Tomcat, APACHE, WebSphere
Cloud Technologies	AWS, Azure, GCP

PROFESSIONALEXPERIENCE:

Client: MedHOK, Tampa, FL

Nov 2024 to Till Date

Role: Sr. Site Reliability Engineer

Responsibilities

- Implemented end-to-end environment setup automation using **Jenkins** pipelines, standardizing application onboarding across multiple client environments. Consolidated manual **AWS CLI** operations and **ad-hoc** scripts into reusable, parameter-driven **Jenkins CI/CD** workflows, improving deployment consistency, traceability, and release velocity.
- Engineered an automated **Schema Compare** and **Flyway** analysis framework to scan **SQL** database migration scripts and classify changes impacting large versus small tables. Integrated metadata-driven table classification sourced from **Amazon S3**, enabling **DBAs** to assess migration risk and execution effort prior to deployment.
- Built **Flyway** automation to split **schema comparison** output into Large and Small **Flyway** artifacts, isolating high-impact database changes from routine modifications. Improved deployment safety by allowing **database** teams to independently estimate execution time and validate large table operations before production rollout.
- Enhanced **schema comparison** reporting by incorporating table size, index count, and total data volume metrics into Large **Flyway** summaries, while filtering out backup and temporary tables using naming conventions such as **_backup** and **_tmp**. Ensured accurate execution estimates and safe production deployments.
- Developed **Jenkins** pipelines for **AWS Lambda** deployments, supporting create and update workflows through configurable inputs such as **AWS** region, **Lambda function** name, and source artifacts stored in **Amazon S3**. Refactored service specific **Groovy** scripts into reusable **Jenkins** pipeline templates.
- Created secure, repeatable **Jenkins** pipelines integrated with **AWS Secrets Manager**, automating creation and updates of sensitive configuration values such as **JDBC** credentials. Enforced environment specific naming conventions and isolation to improve secrets governance and audit readiness.
- Built automation for **AWS Systems Manager (SSM)** Parameter Store using **Jenkins**, enabling centralized creation and updates of environment-specific application parameters. Reduced configuration drift by standardizing parameter hierarchies across **QA** and production environments.
- Designed **Jenkins** based configuration management pipelines to manage application database property files, including automated backups to **Amazon S3** prior to modification. Implemented key validation logic to safely add or update parameters without impacting unrelated configurations.
- Automated **AWS API** Gateway provisioning using **Jenkins**, consolidating documented **AWS CLI** steps into a single parameter driven pipeline. Implemented **IP whitelisting**, environment-specific domain mappings, and stage-level configurations to securely expose internal **web services** with improved auditability.
- Developed reusable **Terraform** modules for private **API Gateway** provisioning, replacing manual setup steps with Infrastructure-as-Code. Enabled consistent, secure, scalable, and compliant **API Gateway** creation across environments to support auto-assign deployment workflows with standardized governance controls.
- Created **Jenkins** pipelines for **Amazon Cognito** app client automation, supporting dynamic scope assignment, environment based naming, and region-specific configurations. Standardized authentication client provisioning for downstream application teams with secure, repeatable, and auditable workflows, reducing manual setup effort.
- Integrated **Datadog APM** across **AWS Lambda** and **API Gateway** based serverless services to capture distributed traces, request latency, error rates, and downstream service dependencies. Correlated application traces with infrastructure metrics to provide end to end visibility into healthcare APIs, significantly improving the root cause analysis.
- Implemented **Datadog** Synthetic Monitoring to continuously validate critical healthcare **API** endpoints and internal service workflows across **QA**, **UAT**, and production environments. Configured multi-step synthetic tests with response validation, latency thresholds, and regional execution to proactively detect availability issues.
- Built comprehensive **Datadog** dashboards aggregating **Lambda** execution metrics, **API Gateway** traffic patterns, error distributions, and deployment health indicators. Enabled real-time operational monitoring for engineering and release teams to quickly assess service behavior during and after **CI/CD** deployments.
- Implemented centralized **Datadog** log ingestion by forwarding **AWS CloudWatch** Logs from **Lambda** functions and **API Gateway** stages into **Datadog**. Enabled log-to-trace correlation using service and request identifiers, allowing faster debugging of runtime errors, misconfigurations, and deployment related regressions.
- Automated **Swagger UI** deployment using **Terraform**, enabling consistent provisioning of **API** documentation endpoints across environments. Integrated **Swagger** with existing **AWS** web services to improve **API** visibility and accessibility.
- Applied **DevOps** best practices by building highly parameterized **Jenkins CI/CD** pipelines, enforcing environment segregation, structured logging, and auditable execution. Reduced manual intervention while improving deployment reliability, release governance, and operational consistency across regulated healthcare systems.

- Maintained secure **CI/CD** guardrails within **Jenkins**, integrating approval gates, structured pipeline logs, and environment-based execution controls to ensure controlled promotions across **QA**, **UAT**, and production environments.
- Integrated **AWS IAM** role-based access controls into **Jenkins** automation to enforce least-privilege execution for pipeline jobs interacting with **API Gateway**, **S3**, **Secrets Manager**, and **SSM**, strengthening overall security posture.
- Standardized **Terraform** remote state management using **S3** backends and state locking, preventing concurrent execution conflicts and configuration drift during multi-environment infrastructure provisioning.
- Implemented pre-deployment validation checks in **Jenkins** pipelines to verify **SSM** parameters, **Secrets Manager** entries, and **API Gateway** readiness before execution. Added structured validation logs to reduce failed deployments.
- Collaborated with database, application, and security teams to align **Flyway** schema automation, **AWS secrets management**, and **API** provisioning workflows with enterprise release and change management standards.
- Integrated **AWS CloudWatch** Logs and metrics into **Jenkins**-driven deployments to capture **Lambda** execution logs, **API Gateway** request metrics, and pipeline outcomes, enabling faster troubleshooting and operational visibility.
- Developed reusable **Groovy** shared libraries in **Jenkins**, version-controlled in **Git** to centralize deployment logic, parameter validation, and error handling, reducing pipeline duplication and improving long-term maintainability.
- Maintained detailed **Jenkins** pipeline documentation and runbooks, outlining execution steps, rollback procedures, and dependency mappings to support on-call engineers and reduce operational knowledge gaps.
- Participated in incident triage and root cause analysis for deployment issues by analyzing **Jenkins** logs, **AWS** service metrics, and configuration changes, contributing to continuous improvement of **CI/CD** automation.

Environment: AWS, Lambda, API Gateway, S3, IAM, Secrets Manager, Systems Manager Parameter Store, CloudWatch, Jenkins, Terraform, Flyway, SQL, Cognito, Swagger, Groovy, Bash, Linux, Git, CI/CD Pipelines, Infrastructure as Code (IaC), Datadog, Cloud Automation, Release Engineering

Client: Walgreens, Deerfield, IL

July 2023 to Oct 2024

Role: Site Reliability Engineer

Responsibilities

- Designed scalable **AWS VPC** architectures with custom route tables, gateways, and subnet level access control configurations. Created **IAM roles** and **Security Groups** to enforce least privilege access, and integrated **Route 53** for intelligent multi-region failover and latency-based routing to support resilient and highly available healthcare workloads.
- Provisioned **EC2** instances across isolated **QA** and **UAT** environments using Auto Scaling Groups and **Elastic Load Balancers**. Integrated **S3** lifecycle policies and Glacier for long-term audit storage of compliance artifacts and healthcare logs, while leveraging **Ansible** for automated server configuration.
- Automated the deployment of secure, highly available **Azure virtual machines** using **Terraform** and **ARM** templates, incorporating compliance agent extensions, disk encryption and secure access policies. Built reusable IaC modules for provisioning resource groups, virtual networks for applying standardized tagging and naming conventions.
- Developed **Python** based automation wrappers for **Terraform** to support dynamic environment creation and validations, such as **DNS** resolution testing, role assumption checks, variable input parsing, enabling faster provisioning of compliant infrastructure across development and staging environments.
- Defined **ECS** task definitions, **IAM** roles, and **Fargate** launch configurations to deploy scalable, stateless microservices for healthcare APIs. Enabled autoscaling, service discovery, and log aggregation using **CloudWatch Logs**.
- Built a scalable and modular configuration management strategy using **Ansible** for **EC2 Linux servers**, automating patching, user management, service hardening, and audit policy enforcement across **AWS** environments. Integrated seamlessly with **AWS Systems Manager** for compliance scanning and secure runbook execution.
- Managed hybrid configuration environments using **Puppet** for **Linux** based systems across **AWS** and **Azure**, ensuring idempotent infrastructure state, consistent patch levels, and centralized configuration enforcement. Enabled secure audit trail logging and compliance alignment for regulated healthcare infrastructure.
- Implemented **GitHub Actions** pipelines to automate linting, vulnerability scans, and multi-environment deployments for **Python** and **Node.js** services. Streamlined approvals, secrets handling, and rollbacks across dev, QA, and production.
- Applied tagging strategies, budget alerts, and cost categorization in **AWS** and **Azure** healthcare environments to chargeback and resource governance models. Integrated **Terraform** state locking and versioning to prevent conflicts.
- Architected and deployed containerized applications on **Azure Kubernetes Service (AKS)**, enabling horizontal pod autoscaling, node pool isolation, and custom taints for production grade and staging microservices.
- Built and versioned **Helm charts** for **Kubernetes** workloads, automating deployment strategies for critical services and internal tools. Configured **Helm** values, parameterized templates, and environment overrides for healthcare specific deployments with clearly separated test, staging, and production configurations.

- Created detailed documentation for **Terraform** modules, **GitHub Actions** workflows, and **ECS** deployment runbooks. Standardized onboarding for DevOps teams with architecture diagrams, **CI/CD** flowcharts, and incident response SOPs.
- Developed predictive **CI/CD** validations by integrating **Python-based** AI models within **Jenkins** pipelines to analyze historical build metrics, reducing failure recurrence and accelerating recovery across regulated release environments.
- Integrated **OpenTelemetry** agents into **AKS** workloads to collect distributed traces, runtime metrics, and structured logs from containerized services. Exported telemetry data to **Azure Monitor** and **Grafana** for end-to-end observability into healthcare transactions, backend workflows, enhancing performance monitoring and issue resolution.
- Used **Kubernetes** Custom Resource Definitions (**CRD**) to configure sidecars for collecting **Prometheus** metrics, **OpenTelemetry** traces, and service-level diagnostics. Applied fine-grained **Kubernetes** network policies to ensure secure traffic segmentation and isolation between healthcare application domains.
- Defined and managed **AWS ECS** services with task definitions, **IAM roles**, and **Fargate** provisioning for scalable, container-based deployment of microservices supporting appointment scheduling and claims data APIs.
- Built secure **CI/CD** pipelines using **Jenkins** and **AWS CodePipeline** for automated code validation and deployment. Integrated **CloudFormation**, **CodeBuild**, and **CodeDeploy** with static analysis, secrets scanning, and manual approvals to protect sensitive healthcare systems and ensure compliant releases.
- Established **blue-green** deployment workflows using **Jenkins** with automated rollback scripts, health checks, and intelligent route switching mechanisms. Enabled seamless, zero downtime releases for critical services such as patient appointment scheduling, pharmacy APIs, and real time healthcare data interfaces.
- Created **Azure DevOps** pipelines with staged approval gates, integrated security scanning tasks, and artifact promotion workflows to streamline release processes. Used **Azure Repos** and **YAML** based pipelines to enforce reproducibility, maintain version control, and ensure consistent application builds across multiple **AKS** environments.
- Built parameterized **Jenkins** pipelines to support multibranch deployments with dynamic inputs and conditional execution stages. Leveraged **AWS** Parameter Store and **Azure Key Vault** integration to securely inject environment specific secrets into build containers, enhancing automation, security, and pipeline flexibility.
- Integrated **Dynatrace One Agent** into containerized applications on **AKS** and **EC2** to capture real-time metrics, distributed traces, and resource utilization. Configured **Synthetic Monitoring** and **Real User Monitoring (RUM)** to simulate user flows, analyze frontend performance, and ensure **SLA** compliance using service-level dashboards.
- Implemented **Datadog** Synthetic Monitoring to simulate critical healthcare workflows such as provider lookup and claim submission, enabling proactive **SLA** compliance tracking. Built real-time dashboards for **JVM** metrics and error rates while integrating alerting with **Slack** and **PagerDuty** for faster response and improved release decision-making.
- Deployed and configured the **Elasticsearch**, **Logstash**, and **Kibana (ELK)** stack using **Chef** to enable centralized log aggregation and search across **AWS EC2** and **Azure VMs**. Implemented index lifecycle management and snapshot policies to optimize storage usage and reduce operational costs for long-term log retention.
- Integrated **ELK** with **AWS CloudWatch** Logs to ingest application logs and **VPC** flow logs, enhancing real-time observability. Automated **ELK** cluster scaling and health checks using **Ansible** playbooks, **Java** based services traceability and audit readiness in healthcare systems.
- Configured **Azure Monitor** diagnostic settings and Log Analytics to track user activity, **API** failures, and system health across services. Integrated Fluent Bit with **OpenTelemetry** to forward logs to **Elasticsearch** and **Azure Monitor**, enabling log-to-trace correlation using injected trace IDs for enhanced audit reporting and observability.
- Designed and implemented secure, scalable **CI/CD** pipelines using **Jenkins** and **AWS CodePipeline**, with detailed documentation of automation scripts, deployment flows, and compliance checkpoints for audit readiness.
- Managed containerized deployments via **Kubernetes (AKS)**, and built production-ready services using **AWS ECS**, enabling scalable, low-latency healthcare **APIs** with efficient resource utilization and fault isolation.
- Applied **AI-enhanced** monitoring practices using anomaly detection in **Dynatrace** and **Datadog** to proactively identify failures and trigger automated healing actions in containerized microservices.
- Managed secrets and environment variables securely using **Azure Key Vault** and **AWS** Parameter Store, implementing **RBAC** and audit logging to monitor access across regulated systems. Enforced encryption, logging, and tagging compliance using **Azure Policy** and **AWS Config**, triggering automated remediation playbooks for drift correction.
- Automated **EC2** and **AKS** provisioning, patching, and service configurations using **Terraform** and **Ansible**, maintaining environment consistency through version-controlled playbooks. Built custom **OpenTelemetry** based logging pipelines to route logs by severity and application tier for streamlined ingestion, compliance validation, and observability.
- Deployed enterprise applications on **AWS Elastic Beanstalk** and **Azure App Service** by configuring environment settings, auto-scaling policies, and deployment slots. Integrated custom domains, **HTTPS** endpoints using **ACM** certificates, and monitored service health through environment metrics to ensure high availability and secure access.

Environment: AWS, VPC, IAM, Route 53, EC2, S3, Glacier, Ansible, Azure, Terraform, ARM templates, Python, ECS, Fargate, CloudWatch Logs, Lambda, SQS, SNS, Puppet, GitHub Actions, CloudFormation, AKS, Helm, Jenkins, CodePipeline, CodeBuild, CodeDeploy, Azure DevOps, Azure Repos, YAML, Parameter Store, Azure Key Vault,

Dynatrace, Synthetic Monitoring, RUM, Datadog, Slack, PagerDuty, ELK Stack, Chef, Log Analytics, Fluent Bit, OpenTelemetry, App Service, Elastic Beanstalk, Azure Monitor, Azure Policy, AWS Config, RBAC, ACM

Client: Humana, Louisville, KY
2023

September 2020 to June

Role: DevOps Engineer

Responsibilities

- Engineered scalable, compliant **Azure** infrastructure using **ARM** templates and **Azure Stack** Services. Automated environment provisioning with reusable **IaC** modules, accelerating deployment timelines, ensuring consistency across environments, and aligning with **HIPAA** audit and compliance standards.
- Implemented enterprise grade **Azure Blob** and **File Storage** solutions for secure healthcare data management. Created storage accounts, configured for content acceleration, set up **SAS** token policies, and managed encryption keys to ensure **HIPAA** compliance and fast data access across distributed systems.
- Architected and deployed highly scalable and reliable infrastructure on **Azure** using reusable **Terraform** modules and **ARM** templates to provision consistent virtual networks, subnets, route tables, and diagnostic settings, ensuring seamless infrastructure parity across development, testing, staging, and production environments.
- Built and maintained internal **Jira/Confluence documentation** to standardize CI/CD, automation scripts, IaC modules, and service ownership playbooks, significantly reducing DevOps onboarding time and audit review efforts.
- Containerized **Java** based microservices using **Docker** and deployed them on **Azure Kubernetes Service (AKS)**, implementing horizontal **pod** autoscaling and dedicated node pools to ensure high scalability, resource isolation, and fault tolerance across test and production grade clusters.
- Integrated **AI-powered** automation to validate **Terraform** configurations and detect anomalies in **CI/CD** pipelines. Reduced manual scripting by 40% through predictive error detection, environment templating, and self-healing logic.
- Designed a full-stack observability framework using **OpenTelemetry**, **Prometheus**, **Datadog**, and **ELK** to correlate logs, metrics, and traces across **Kubernetes** workloads. Enabled RCA and enhanced **SLO** monitoring with alert routing.
- Implemented serverless architecture using **Azure Functions**, **Service Bus**, and Event Grid to trigger real-time billing workflows and appointment reminders, improving latency and reducing costs for infrequent workloads.
- Integrated **Red Hat Advanced Cluster Management (ACM)** for Azure Kubernetes Service **AKS** environments to centralize policy governance, enforce healthcare compliance, and provide unified visibility across multiple clusters. Enabled consistent configuration management and improved multi-cluster observability across patient care systems.
- Leveraged **Helm charts** and **Kubernetes** manifests to manage dynamic and environment specific resource configurations efficiently. Enabled seamless zero-downtime deployments using **Kubernetes** rolling update strategies and implemented automated rollback mechanisms triggered by health check failures to maintain application availability.
- Developed secure, multi-stage **CI/CD** pipelines in **Azure DevOps** for scalable **Maven** based **Java** applications, integrating secrets and credentials from **Azure Key Vault** and deploying versioned artifacts to **Nexus** Repository. Used custom **PowerShell** scripts to promote packages across environments and support **blue-green** deployments.
- Integrated **Jenkins** with **Terraform** and **Ansible** workflows to automatically provision complete test environments before pipeline execution. Enabled developers to quickly launch end-to-end test beds for eligibility **API** microservices in under 15 minutes, improving testing efficiency and delivery speed.
- Integrated **GitLab CI** pipelines for automated testing, artifact promotion, and provisioning using **Terraform** and **Ansible**, ensuring policy-compliant deployments and consistent state management across **Azure Kubernetes** clusters.
- Led initiatives to create standardized, version-controlled templates and detailed documentation for **ARM**, **Terraform**, and **Ansible**-based deployments across **HIPAA**-compliant **Azure** environments.
- Created **AI**-integrated scripts to analyze infrastructure logs and predict scaling needs, feeding insights into **Ansible** patch automation routines for 500+ **VMs**.
- Embedded **Checkmarx** static code analysis into **CI/CD** pipelines, configured scan thresholds, and generated compliance reports, enabling secure releases and protection against **OWASP** vulnerabilities in healthcare systems.
- Authored end-to-end **DevOps** process documentation including **CI/CD** pipelines, rollback strategies, **Helm** chart configurations, and secure credential handling with **Key Vault** and **Parameter Store**.
- Deployed **OpenTelemetry** agents across **AKS** workloads and integrated telemetry into **Azure Monitor** and **AppDynamics**. Instrumented real-time tracing of patient eligibility calls, improving MTTR by 30% for API failures.
- Built comprehensive **AppDynamics** dashboards for **JVM** performance monitoring of provider search microservices running in production environments. Tuned garbage collection settings and optimized connection pool sizing based on detailed APM metrics to reduce latency spikes and improve application responsiveness during high traffic periods.

- Authored reusable **Terraform** scripts for provisioning **Azure** resources, including virtual networks, key vaults, private endpoints, and diagnostics settings with tagging standards. Ensured consistent and reproducible infrastructure deployments across multiple teams supporting high-availability claims and eligibility **API** workloads.
- Developed **Ansible** playbooks for OS patching, VM bootstrap, and middleware configuration across **Ubuntu** and **RHEL** workloads in **Azure**. Enabled patch automation for 500+ servers supporting health care enterprise tools.
- Integrated **Puppet** manifests within **Jenkins** pipelines for configuration drift enforcement. Used **Hiera** and custom facts to ensure system state consistency across on-prem and cloud servers.
- Created **Python** automation scripts to seamlessly integrate with **Puppet APIs** and **Jenkins** webhooks, enabling batch deployments of **Java** WAR files to multiple test environments. Implemented automated verification of post-deployment health using API probes, ensuring application stability and deployment success across environments.
- Maintained structured **Git** based branching strategies across enterprise **GitHub** and **Azure** Repos to support collaborative development workflows. Enforced trunk-based development practices, implemented pull request validations, and code review policies to ensure high-quality code integration with regulated healthcare release cycles.
- Migrated legacy applications from **SVN** and **TFS** source control systems to **Git**, resolving complex branching inconsistencies and accurately mapping user contribution histories. Modernized source code management (**SCM**) practices by establishing **Git** workflows that enhanced collaboration, traceability, and continuous integration readiness.
- Configured **Azure Container Registry** (ACR) pipelines for **Docker** image builds and integrated image scanning tools to detect vulnerable base images during builds. Managed **Jenkins** agents on **Azure** VMs with label-based execution for **Linux** and **Windows** builds, optimizing performance by tuning **JVM** heap settings.
- Used **JIRA** to manage **agile** sprints, track defect resolution and ensure timely delivery of key features. Linked **JIRA** issues to **CI/CD** pipeline deployments, enabling complete traceability and compliance tracking across environments.
- Tuned **SQL** queries embedded within **Java** based applications by analyzing slow query logs, optimizing query execution plans, refactoring complex joins, and introducing appropriate indexing strategies.
- Achieved a 40% improvement in dashboard load times for the provider analytics platform, significantly enhancing user experience and system responsiveness under peak load conditions. Developed **Python** based batch loaders to process bulk data into **Azure SQL** and **PostgreSQL**, supporting data refreshes for claims reporting dashboards.
- Integrated **RESTful APIs** with **Azure** Logic Apps to automate claims adjudication workflows and initiate real-time notifications to downstream systems. Streamlined inter system communication processes, reduced dependency on manual intervention, and enhanced operational efficiency in claims processing pipelines.

Environment: Azure, ARM templates, Azure Stack, Blob Storage, Terraform, Virtual Networks, Jira, Confluence, Docker, AKS, OpenTelemetry, Prometheus, Datadog, ELK, Azure Functions, Service Bus, Event Grid, Red Hat ACM, Kubernetes, Helm, Azure DevOps, Key Vault, Nexus Repository, PowerShell, Jenkins, Ansible, GitLab CI, ECS, Checkmarx, Parameter Store, AppDynamics, Ubuntu, RHEL, Puppet, Python, API Probes, GitHub, Azure Repos, SVN, TFS, Azure Container Registry (ACR), JIRA, SQL, Azure SQL, PostgreSQL, Logic Apps, REST APIs

Client: Bank of America, Charlotte, NC
2020

November 2017 to August

Role: DevOps Engineer

Responsibilities:

- Designed and deployed highly available infrastructure on **Google Cloud Platform (GCP)** using **Compute Engine**, **Cloud SQL**, and **Cloud Storage** to support mission-critical financial applications across business units. Ensured fault tolerance, data redundancy, and alignment with internal SLAs, compliance frameworks, and audit standards.
- Automated **GCP** resource provisioning with **Terraform** by developing modular, reusable templates for **VPC networks**, **subnets**, **IAM roles**, and **firewall** rules.
- Authored modular **AWS CloudFormation** templates for provisioning **VPCs**, **IAM** roles, **RDS** clusters, and security policies. Used change sets and drift detection to ensure audit-compliant environments across UAT and production.
- Architected secure, serverless GCP workloads using **Cloud Functions**, **Pub/Sub**, and **Cloud Scheduler** to automate reconciliation reports and customer notification systems in a cost-efficient, event-driven manner.
- Architected and maintained resilient cloud infrastructure on **AWS** using **EC2**, **S3**, **RDS**, and **IAM** within secure **VPCs**. Supported scalable banking systems while enforcing multi-AZ redundancy.
- Built proactive monitoring and alerting for **GKE**-hosted services using **OpenTelemetry**, **Prometheus**, and alert routing via **Slack** and **PagerDuty**, enhancing observability and reducing MTTR for customer-facing applications.
- Built **AWS** infrastructure automation using **Terraform** by creating version-controlled modules for networking and IAM. Configured **EC2** autoscaling with **CloudWatch** metrics to ensure availability and performance during peak financial cycles while reducing manual provisioning and improving audit readiness.
- Deployed **Docker** and **Podman** containers across multi-platform **Kubernetes** ecosystems including **GKE**, **OpenShift**, and **Rancher**. Implemented container lifecycle management strategies, optimized image builds and enforced runtime security policies. Enabled infrastructure consistency and portability across hybrid cloud environments.

- Developed and maintained **Helm charts** for scalable deployment of microservices on **Kubernetes** and **OpenShift** platforms, incorporating best practices for templating, environment specific configuration, and rollback strategies.
- Engineered containerized application delivery pipelines on **AWS** and **Kubernetes**, leveraging **EKS** native features, **IAM** roles, **VPC** security controls, and **CloudWatch** monitoring to ensure secure, observable, and scalable infrastructure for microservices deployment across production environments.
- Implemented custom **networking architectures** in GCP and AWS with **subnets**, **firewall rules**, **private link endpoints**, and **DNS-based failover**, enforcing compliance and isolation for high-security financial data environments.
- Automated the provisioning of **Kubernetes** clusters using **Terraform** and **AWS CLI**, implementing **RBAC** policies, namespace resource quotas, and integrating **DevSecOps** toolchains for vulnerability scanning, image signing, and compliance enforcement to support secure, auditable **CI/CD** workflows.
- Maintained **CI/CD** pipelines using **Jenkins** and **Google Cloud Build**, integrating with **GitHub**, **GCP** services, and **Datadog APM** and logging hooks to enhance build-test-deploy observability. Reduced deployment time by 50% while gaining actionable insights on failure patterns and runtime behavior.
- Integrated **Datadog agents** and **Kubernetes** monitors to track resource utilization, pod-level health, and service metrics across GKE. Built dashboards, alerts to detect infrastructure anomalies and application slowdowns in real-time.
- Developed **Splunk** dashboards and custom queries to monitor KPIs, detect anomalies, and analyze log data for system health, performance and security insights, enabling proactive incident response and improved operational intelligence.

Environment: GCP, Compute Engine, Cloud SQL, Cloud Storage, Terraform, VPC, IAM, Firewall Rules, CloudFormation, AWS, EC2, S3, RDS, Cloud Functions, Pub/Sub, Cloud Scheduler, OpenTelemetry, Prometheus, Slack, PagerDuty, CloudWatch, GKE, OpenShift, Docker, Podman, Rancher, Kubernetes, Helm, CI/CD, EKS, DNS, AWS CLI, RBAC, Jenkins, Google Cloud Build, GitHub, Datadog, Splunk

Client: AT&T, Dallas, TX
2017

December 2015 to October

Role: UNIX/Linux Administrator

Responsibilities:

- Monitored system performance and resource utilization on **UNIX/Linux** servers using **Nagios** and **Munin**, proactively identifying bottlenecks, resolving issues, and ensuring system health and uptime across production environments.
- Developed and maintained robust **Bash** and **PowerShell** Core scripts to automate repetitive administrative tasks including file system backups, log rotation, user management, and cross-platform configuration.
- Version-controlled **Bash** and **PowerShell** scripts using **Git**, enabling collaborative development, rollback capability, and standardized configuration management across **UNIX/Linux** systems.
- Managed software installations, security patches, and updates via **yum** and **apt-get**, maintaining consistent package versioning across **UNIX** and **Linux** distributions while ensuring compliance with security standards.
- Implemented automated performance tracking on **UNIX** and **Linux** environments by creating **Bash-based** monitoring scripts for system metrics like disk and memory usage integrating **cron jobs** and email notifications.
- Troubleshoot network, and application issues across **UNIX/Linux** and **Windows** environments, minimizing downtime and enhancing service continuity through systematic root cause analysis and issue resolution strategies.
- Collaborated with senior **UNIX/Linux** administrators to define and enforce system policies, access control standards, and security procedures, contributing to a stable and compliant enterprise environment.
- Provisioned and supported web servers (**Apache**, **Nginx**) and database servers (**MySQL**, **PostgreSQL**), optimizing performance, securing configurations and ensuring high availability for critical business applications.
- Configured and administered core network services including **DNS**, **DHCP**, **NFS**, and **Samba**, ensuring reliable connectivity, streamlined resource sharing, and consistent network performance across distributed systems.

Environment: UNIX, Linux, Nagios, Munin, Bash, PowerShell, Apache, Nginx, Corn, MySQL, PostgreSQL, Yum, apt-get, VMware, VirtualBox, DNS, DHCP, NFS, Samba, Git

Client: Zoho, Chennai, India
2015

May 2014 to October

Role: Software Engineer

Responsibilities:

- Assisted in developing dynamic web applications using the Struts **MVC** framework, contributing to modular design by separating business logic from UI for better code organization and maintainability.

- Supported integration of **SOAP** based **XML** web services to enable cross-system communication, working on data parsing and transformation to ensure consistent and accurate message formats.
- Contributed to building reusable components using **JSP**, **Servlets**, **JSTL**, and custom tag libraries, helping reduce redundancy and simplify code updates during the development cycle.
- Worked on front-end development tasks using **HTML5**, **CSS3**, and **Bootstrap**, helping create responsive UI screens optimized for desktops and mobile devices through media queries and flexible layouts.
- Assisted in creating mobile-friendly web interfaces using **Bootstrap** UI elements like modals and forms, improving usability and reducing design time through reusable components.
- Gained hands-on experience with **Git** and **Jenkins**, participating in version control, code merges, and automated builds within the **CI/CD** pipeline for improved collaboration and deployment consistency.
- Helped configure **Maven** based builds, learning to manage library dependencies, plugin configurations, and automate the full project build lifecycle by editing **POM** files under supervision and guidance.
- Wrote basic **SQL** queries and stored procedures and worked with **JDBC** to connect Java applications to relational databases, supporting reliable data access and ensuring functional backend integration.

Environment: Java, Java EE, Struts, Maven, JDBC, JSP, Java Servlets, JSTL, SOAP, HTML, CSS, XML, Bootstrap, Jenkins, Git, JIRA, SQL

EDUCATION:

Vellore Institute of Technology
Bachelor of Technology

June 2010 to April 2014
Computer Science

CERTIFICATIONS :

- **Salesforce Certified Sales Cloud Consultant**
- **Marketo Certified Solutions Architect (MCSA)**
- **HubSpot RevOps Professional Certification**
- **Pragmatic Institute: Certified Product Marketing Executive (Level VI)**
- **Section4: Certified Growth & Brand Strategist**
- **Google Ads: Search, Video & Display Professional Certification**